

CUSTOMER FEATURE GUIDE

Veyron

Kubernetes-native VM command center.

Declarative VM provisioning, day-2 operations, and a Mission Control dashboard for KubeVirt — forged in Rust.

by ZyvorAI Labs

What is Veyron?

Veyron turns verbose KubeVirt YAML into a single, opinionated command center for virtual machines on Kubernetes. It ships four surfaces over one Rust core — a CLI, a k9s-style TUI, a CloudOS web dashboard, and a REST/WebSocket API — plus a Go operator that makes VMs GitOps-native through the VeyronVM custom resource. From a 44-image template catalog and multi-VM blueprints to browser consoles, snapshots, live migration, SOC detections, and an AI copilot, Veyron covers the full VM lifecycle from create to retire.

44

OS templates across 15 families

40+

dashboard pages

4

surfaces: CLI, TUI, Web, API

8

resource profiles

What's inside this guide

Getting started — access & first workflows	start here
1 · VM Provisioning & Templates	7 features
2 · Consoles & Guest Access	5 features
3 · Snapshots & Data Protection	5 features
4 · Migration & High Availability	5 features
5 · Storage & Disks	4 features
6 · Networking	4 features
7 · AI Copilot — Ask Zeus	5 features
8 · Security & Compliance	5 features
9 · Cost & FinOps	4 features
10 · Observability & Monitoring	4 features
11 · GitOps & Operator	5 features
12 · Platform & Surfaces	5 features
Support & contact	→

Access & first workflows

How to reach Veyron and get your first result. Assumes it has been deployed for you.

How to access it

WEB

CloudOS "Mission Control" dashboard (40+ pages, Spotlight `⌘K`, dock, Finder sidebar). Start the server with `veyron serve`, then open `https://localhost:8080` (redirects to `/dashboard`).

CLI

The `veyron` binary drives everything headless — e.g. `veyron create web-01 --template ubuntu-22.04`, `veyron blueprint deploy lamp`, `veyron gitops-export`. Run `veyron commands` for the full 200+ subcommand surface, or `veyron tui` for the k9s-style terminal UI.

API

REST + WebSocket API served by `veyron serve` under `/api/v1` (with `/api/v2` and a `/api/v1/veyron` product alias); embedded OpenAPI at `/api/openapi.json` (or `veyron api-spec`). List live routes with `veyron api-routes`. VNC/serial/RDP consoles and the metrics stream tunnel over WebSockets.

LOGIN

Automation uses API keys — `export VEYRON_API_KEYS="admin:supersecret,write:devkey,readonly:viewkey"` then send `Authorization: Bearer <key>`. Human login is OIDC SSO (PKCE) via `VEYRON_OIDC_ISSUER` / `VEYRON_OIDC_JWKS_URL`, with groups `veyron-admin` / `veyron-write` mapped to roles; WebSocket consoles authenticate with a one-time ticket from `POST /api/v1/ws/ticket`.

NEEDS

A KubeVirt-enabled Kubernetes cluster reachable via `KUBECONFIG` (Veyron is a control plane over the KubeVirt/K8s APIs).

Your first workflows



1. Browse the catalog: `veyron templates` (and `veyron template ubuntu-22.04` for details).
2. Create it: `veyron create web-01 --template ubuntu-22.04 --cpus 4 --memory 8Gi -n dev`.
3. Watch it come up: `veyron status web-01 --watch`.
4. Reach the guest: `veyron console web-01 -n dev` (or open the VNC console from the dashboard VMs page).



1. List stacks: `veyron blueprints` and inspect one with `veyron blueprint lamp`.
2. Preview the generated resources: `veyron deploy lamp --prefix prod --dry-run`.
3. Deploy and start in dependency order: `veyron deploy lamp --prefix prod --start`.
4. Verify: `veyron ls -A`.



1. Export live VMs to VeyronVM CR manifests: `veyron gitops-export --directory gitops/`.
2. Commit the manifests and reconcile them with Argo CD / Flux (operator installed from `charts/veyron-operator`).
3. Detect divergence over time: `veyron gitops-diff --directory gitops/` and `veyron gitops-status` (or the `veyron drift` view).



1. Take an app-consistent snapshot: `veyron snapshot-create my-vm --name backup-20260417` (guest filesystems are frozen via the QEMU guest agent).
2. Make the change, then confirm the copy: `veyron snapshot-get backup-20260417`.
3. Roll back if needed: `veyron snapshot-restore backup-20260417 --target restored-vm --start`.



1. Preview per-VM moves: `veyron evacuate-node worker-01 --plan`.
2. Live-migrate a single VM if needed: `veyron migrate my-vm --plan` then execute.
3. Track progress: `veyron migration-status my-vm --watch` and `veyron evacuation-status`.



1. Set OIDC env on the API deployment (`VEYRON_OIDC_ISSUER` , `VEYRON_OIDC_JWKS_URL`) and map IdP groups `veyron-admin` / `veyron-write`.
2. Start the server: `veyron serve` , then sign in through your IdP at `/dashboard`.
3. Apply a `VeyronPolicy` CRD (`kubectl apply -f policy.yaml`) with CEL deny rules so non-compliant VM creates are rejected at admission.

VM Provisioning & Templates

From a blank cluster to a running fleet without hand-writing manifests.



Declarative VM Builder

Create KubeVirt VirtualMachines from the CLI, wizard, or dashboard with CPU, memory, disk, storage class, and cloud-init.

Skip error-prone YAML — describe the VM you want and Veyron generates a validated manifest.

How · CLI: `veyron create web-01 --template ubuntu-22.04 --cpus 4 --memory 8Gi`. Web: VMs page → New VM. TUI: `veyron tui`. CRD: `kubectl apply -a VeyronVM (veyron.io/v1alpha1)` for the operator to reconcile.



44-Image OS Catalog

Launch from 44 curated templates spanning 15 OS families — Ubuntu, Fedora, Debian, RHEL, Rocky, AlmaLinux, Windows, and BSD.

Battle-tested base images with sensible defaults for every common guest.

How · CLI: `veyron templates` to list, `veyron template ubuntu-22.04` for details. Web: Catalog page. Then reference the name via `--template`.



Multi-VM Blueprints

Deploy whole stacks — LAMP, a Kubernetes cluster, 3-tier apps, CI/CD — as one dependency-ordered blueprint.

Stand up an entire environment with a single deploy instead of provisioning VMs one at a time.

How · CLI: `veyron blueprints`, then `veyron deploy lamp --prefix prod --start` (`--dry-run` to preview). Web: Blueprint Studio. CRD: apply a VeyronBlueprint for the operator.



Resource Profiles

Apply prebuilt sizing profiles (dev, test, prod, high-perf, microservice) or author your own with custom CPU topology.

Consistent, right-sized VMs across teams without re-deciding cores and memory each time.

How · CLI: `veyron profiles`, `veyron profile database`, `veyron recommend database --alternatives`; author with `veyron profile-create`. Apply on create via the profile flag.



Interactive Wizard & Batch

Walk through guided VM creation, or provision many VMs at once from a batch YAML/JSON file.

One prompt-driven flow for newcomers, one file for bulk fleet builds.

How · CLI: `veyron wizard my-vm` (alias `veyron wiz`) for the guided flow; `veyron batch some-batch.yaml --dry-run` for bulk. Web: New VM wizard.



Clone, Export & Dry-Run

Duplicate an existing VM, export any VM as a manifest, or preview the generated resource before it touches the cluster.

Reproduce known-good VMs and review exactly what will be applied first.

How · CLI: `veyron clone source-vm cloned-vm --start`; `veyron generate my-vm --template fedora-40 --kubvirt` (alias `veyron gen`) to export; add `--dry-run` to any create/deploy.



GPU VMs (Passthrough)

Attach whole NVIDIA GPUs to VMs via PCI passthrough with automatic driver-friendly settings, a per-node GPU inventory, and a create-time check that the cluster can satisfy the request.

[Run CUDA, rendering, or AI workloads in isolated VMs on your own GPUs — no licensing required.](#)

How · CLI: `veyron create gpu-01 --template ubuntu-22.04 --gpu 1` . API: `POST /api/v1/vms` with `"gpu": {"count": 1}` . Web: Forge wizard GPU picker; Nodes page → GPU Inventory. Passthrough GPU VMs move via stop/start — live migration is physically impossible and Veyron says so up front.

Templates resolve from an embedded catalog for offline CLI use and from cluster VMTemplate/VMProfile CRDs for GitOps — the operator and CLI agree on names.

Consoles & Guest Access

Reach any VM directly from the browser or terminal — no virtctl timeouts.



Browser VNC Console

Open a graphical VNC session to any VM over a direct Kubernetes WebSocket, right in the dashboard.

Full desktop access without VPNs, jump hosts, or virtctl port-forwards that time out.

How · Web: VMs page or Console Hub → VNC. CLI: `veyron vnc web-01 -n dev` . API: `GET /api/v1/vms/:ns/:name/vnc` (WebSocket, ticket-authed).



Serial Console & Logs

Attach to a VM's serial console and stream logs from its virt-launcher pod with `follow` and `tail`.

Diagnose boot problems and kernel panics where a graphical console can't reach.

How · CLI: `veyron console web-01 -n dev` ; stream logs with `veyron logs` (alias `veyron log`). Web: Console Hub → Serial. API: `GET /api/v1/vms/:ns/:name/serial` .



RDP Screen Sharing

Expose and connect to Windows guests over RDP through the dashboard's Screen Sharing surface.

Native Windows remote-desktop access managed from the same console as your Linux VMs.

How · Web: Screen Sharing / Console Hub. API: `POST /api/v1/vms/:ns/:name/rdp-expose` . Gated by `VEYRON_ALLOW_PUBLIC_RDP` .



SSH & Key Injection

SSH into running VMs and inject public keys via KubeVirt accessCredentials, propagated through the guest agent or configDrive.

Provision secure key-based access to a VM without rebuilding or manual guest edits.

How · CLI: `veyron ssh web-01` ; inject a key with `veyron ssh-key-inject` . Keys can also be supplied via cloud-init on create.



Guest Agent Signals

Read QEMU guest-agent data — in-guest filesystem usage, network info, and health signals — from CLI or dashboard.

See what's actually happening inside the guest, not just what Kubernetes reports outside it.

How · CLI: `veyron health web-01 --detailed` , `veyron disk-health web-01` . API: `GET /api/v1/vms/:ns/:name/guest/status` . Web: VM detail / VM Capsule. Requires the QEMU guest agent running.

Snapshots & Data Protection

Application-consistent point-in-time copies, backups, and disaster-recovery plans.



VM Snapshots

Create, list, inspect, delete, and restore KubeVirt VirtualMachineSnapshots, including in-place restore.

Roll a VM back to a known-good state in seconds before risky changes.

How · CLI: `veyron snapshot-create my-vm --name backup-20260417`, `veyron snapshot-list`, `veyron snapshot-restore backup-20260417 --target restored-vm --start` . Web: Snapshots page. API: `/api/v1/snapshots` .



App-Consistent Freeze

Quiesce guest filesystems with fsfreeze via the QEMU guest agent before snapshotting, then thaw them.

Snapshots that databases and apps can actually be restored from — not just crash-consistent copies.

How · Applied automatically during snapshot; drive manually with CLI `veyron guest-freeze` / `veyron guest-unfreeze` , or API `POST /api/v1/vms/:ns/:name/guest/freeze` .



VM Backups

Take full, incremental, or differential backups with gzip/zstd/lz4 compression and integrity verification.

Portable, verifiable VM copies with scheduled full/incremental rotation.

How · CLI: `veyron backup-create my-vm --name nightly-001`, `veyron backup-verify nightly-001 --verification-type full`, `veyron backup-restore nightly-001 --target restored-vm --start` . Web: Backups page.



Backup Schedules

Define hourly, daily, weekly, or monthly backup schedules scoped to specific VMs or the whole fleet.

Set-and-forget protection so no VM drifts out of your backup window.

How · CLI: `veyron backup-schedules` . Scheduled snapshots via API `/api/v1/snapshot-schedules` (ConfigMap-driven scheduler). Web: Backups page.



Velero & DR Plans

Surface Velero Backup/Restore status and author disaster-recovery plans with dry-run execution and failover.

Cluster-level recovery readiness alongside per-VM backups, tested before you need it.

How · CLI: `veyron velero-dr` . Web: DR page (dry-run + failover). Requires Velero installed in the cluster.

Migration & High Availability

Move workloads between nodes and keep critical VMs running through maintenance.



Live & Offline Migration

Migrate VMs across nodes using live, offline, or post-copy strategies, with a plan preview before executing.

Rebalance load or evacuate hardware without downtime for migratable workloads.

How · CLI: `veyron migrate my-vm --plan` then execute. API: `POST /api/v1/vms/:ns/:name/migrate` . Web: Scheduling page.



Node Evacuation & Drain

Cordon, uncordon, and evacuate nodes with controlled parallel migrations, timeouts, and status tracking.

Take a node down for maintenance while VMs safely relocate themselves.

How · CLI: `veyron evacuate-node worker-01 --plan` , then `veyron evacuation-status` . Web: Nodes / Scheduling page.



VM High Availability

Configure per-VM HA priority and eviction strategy (live-migrate, shutdown, none) and view HA status.

Guarantee that your critical VMs are the ones that survive node failures.

How · CLI: `veyron ha-config my-vm --enable --priority critical --eviction-strategy live-migrate` ; check with `veyron ha-status` .



Migration Tracking & Policies

Watch live migration progress and manage KubeVirt migration policies from the scheduling surface.

Know exactly where a migration is and enforce fleet-wide migration guardrails.

How · CLI: `veyron migration-status my-vm --watch` , `veyron migration-list -A` . Web: Scheduling page. API: `GET /api/v1/vms/:ns/:name/migrations` .



Migration Eligibility Guard

Every migration request is pre-checked: passthrough GPUs, stopped instances, and KubeVirt's own non-migratable verdicts are refused up front with the exact reason and the recommended alternative.

No migrations that silently spin forever — you learn immediately why a VM can't move and what to do instead.

How · Automatic on every migrate: the API returns a structured 409 with `blockers[]` ; the dashboard shows the reason as a toast. Bypass with `"force": true` .

Storage & Disks

Grow, hotplug, and monitor VM disks without downtime.



Online Disk Expansion

Expand VM disks and resize PVCs, with a plan preview and generated in-guest filesystem-grow scripts.

Add capacity to a running VM instead of rebuilding it when it fills up.

How · CLI: `veyron disk-expand my-vm rootdisk 100Gi --plan`, then run the `veyron disk-script --filesystem lvm --device /dev/vda` output in-guest. PVC resize via API `PATCH /api/v1/storage/pvcs/:ns/:name` (also in the Storage UI).



Volume Hotplug

Attach and detach PVCs on a running VM via `virtctl addvolume/removevolume`.

Give a live VM more storage on demand — no restart required.

How · CLI: `veyron volume-add / veyron volume-remove`. API: `POST /api/v1/vms/:ns/:name/volumes/hotplug` (and `hotremove`, `status`).



Disk Health & Usage

Report per-VM disk health and usage statistics, sorted by utilization, size, or free space.

Spot the disks about to fill before they take an application down.

How · CLI: `veyron disk-health my-vm --detailed`, `veyron disk-usage`. Web: Storage page.



Storage Pools & PVCs

Browse PVCs, storage classes, and storage pools across the cluster from the Storage page.

One inventory of every volume backing your VM fleet.

How · Web: Storage page. API: `GET /api/v1/storage/pvcs`, `GET /api/v1/storage/classes`.

Networking

Per-VM connectivity, policy enforcement, and network intelligence.



Interfaces & Bandwidth

List VM network interfaces, monitor per-interface bandwidth in watch mode, and analyze traffic and top talkers.

See which VM is saturating the network and on which interface.

How · CLI: `veyron network-list my-vm`, `veyron network-bandwidth my-vm --watch`, `veyron network-traffic my-vm --period 1h`. Web: Network Intel page.



Multus Multi-Network

Attach additional Multus networks to a VM and manage NetworkAttachmentDefinitions.

Give VMs the multiple NICs and VLANs that real workloads need.

How · CLI: `veyron network-get my-vm eth0` to inspect interfaces; declare extra networks in the VM's interface config. Web: Network Intel page.



Network Policies & Egress

List and inspect NetworkPolicies, and apply a default internet-egress policy on VM create (opt-out available).

VMs come locked down by default instead of wide open to the internet.

How · CLI: `veyron network-policies -A`, `veyron network-policy` for detail; egress policy applied automatically on `veyron create` (opt-out flag).



Cilium & PacketWolf

View Cilium agent status and policies, with an optional PacketWolf network-intelligence integration.

Deeper network visibility for clusters running Cilium or the PacketWolf brain.

How · CLI: `veyron cilium`. Web: Cilium / Network Intel pages. Enable PacketWolf via `VEYRON_PACKETWOLF_*` env.

AI Copilot — Ask Zeus

Plain-language diagnosis with cluster evidence and one-click fixes.



Ask Zeus Assistant

Ask VM questions in natural language (🗨️) and get root cause, cluster evidence, and recommended actions.

Skip the stack traces — describe the problem and get a grounded answer.

How · Web: Ask Zeus page (🗨️). Works deterministically without an LLM; connect an OpenAI-compatible model via `VEYRON_AI_*` for richer conversation.



Veyron Doctor

Generate a per-VM health score with prioritized issues drawn from live KubeVirt/Kubernetes state.

A quick triage read on any VM before you dig into it manually.

How · CLI: `veyron doctor (fleet) or veyron health my-vm --detailed`. Web: Insights page. API: `GET /api/v1/vms/:ns/:name/guest/doctor`.



Scheduling Explainer

Translate a Pending or Unschedulable VM into a human-readable reason and fix.

Understand why a VM won't start without decoding scheduler events yourself.

How · Web: Scheduling page / Ask Zeus. CLI: surfaced through `veyron doctor` and `veyron placement my-vm --strategy leastloaded`.



Error Explainer & YAML Builder

Turn cryptic K8s/KubeVirt errors into plain English and generate cluster-validated VirtualMachine YAML.

Get unblocked on obscure errors and hand off valid manifests in one step.

How · Web: Ask Zeus. CLI: generate validated YAML with `veyron generate my-vm --template fedora-40 --kubevirt` and check it with `veyron validate examples/basic-vm.yaml`.



Lens Advisors

Purpose-built advisors — Network Lens, Storage Doctor, Backup Advisor, Cost Advisor, Security Sentinel — inspect a VM or the whole fleet.

Domain-specific expertise on connectivity, bloat, coverage gaps, spend, and exposure.

How · CLI: `veyron insights-generate my-vm, veyron recommendations --category cost --with-savings`. Web: Insights page.

The v1 assistant is deterministic and works without any external LLM by composing real cluster data; connect an OpenAI-compatible model (VEYRON_AI_*) to enable richer conversational answers.

Security & Compliance

Detections, hardening, policy enforcement, and an audit trail for the VM fleet.



SOC & Detections

Run a security operations center over VM activity — detections, threat hunts, attack-surface mapping, and SIEM export.

Turn VM events into actionable security signal, pushed to Elastic or your SIEM.

How · Web: SOC page. CLI: `veyron security-assess my-vm`, `veyron security-scan my-vm --scan-type standard`. SIEM export configured via integration env (Splunk/QRadar/Sentinel/Elastic).



Hardening Profiles

Scan VMs and apply hardening baselines — CIS, STIG, PCI-DSS, NIST — with verify-only mode.

Bring VMs up to a recognized security standard on demand.

How · CLI: `veyron security-harden my-vm --profile cis --verify-only`. Web: Security page.



Compliance Frameworks

Check VMs against PCI-DSS, HIPAA, SOC 2, ISO 27001, GDPR, NIST, and CIS, and produce reports.

Evidence for auditors without manual per-VM checklists.

How · CLI: `veyron compliance-check my-vm --framework cis`. Web: Compliance page.



Policy Enforcement

Define VeyronPolicy CRDs with CEL deny rules so violating VMs never make it into the cluster.

Stop misconfigured or non-compliant VMs at admission instead of cleaning up after.

How · CRD: `kubectl apply` a VeyronPolicy (`veyron.io/v1alpha1`) with CEL deny rules; the Go operator enforces them. Web: Policies page.



RBAC, Audit & SSO

Inspect cluster RBAC roles and bindings, review a namespace-aware audit trail, and sign in via OIDC SSO.

Know who did what, and gate access to identity your org already trusts.

How · CLI: `veyron audit-list my-vm`. Web: RBAC / Audit pages. SSO via `VEYRON_OIDC_*` env and `GET /api/v1/auth/oidc/config` (PKCE token exchange at `POST /api/v1/auth/oidc/token`).

Cost & FinOps

Understand, budget, and trim what your VM fleet spends.



Cost Analysis

Break down fleet and per-VM cost by namespace, team, or project over configurable time windows.

[See where the money goes across every VM you run.](#)

How · CLI: `veyron cost-analyze --period 30d`, `veyron cost-summary --group-by namespace`. Web: Costs page. Wire real numbers with `VEYRON_OPENCOST_URL`.



Budgets & Alerts

Create budgets scoped to global, namespace, team, or project with configurable alert thresholds.

[Get warned before spend blows past what you planned for.](#)

How · CLI: `veyron budget-create team-a --amount 500 --period monthly --scope namespace:production`, `veyron budget-list`. Web: Costs page.



Waste & Optimization

Surface idle, oversized, storage, and snapshot waste, and get prioritized cost-optimization recommendations.

[Reclaim spend from the VMs and volumes nobody is using.](#)

How · CLI: `veyron recommendations --category cost --with-savings`. Web: Costs / Insights pages.



Cost Forecasting

Project 7/30/90-day costs and compare them against a budget.

[Plan next quarter's VM spend before it arrives.](#)

How · CLI: `veyron forecast`, `veyron cost-report --report-type monthly --format json`. Web: Forecasting page.

Observability & Monitoring

Live metrics, hotspots, SLOs, and stack discovery across the fleet.



Live VM Monitoring

Stream live CPU/memory/disk stats for a VM, compare multiple VMs, and rank the top resource consumers.

[Watch a VM in real time and instantly find the fleet's heaviest workloads.](#)

How · CLI: `veyron monitor-live`, `veyron monitor-compare`, `veyron monitor-top` (also `veyron top / resources`). Web: Monitoring page. API: `GET /api/v1/ws/metrics` (WebSocket).



Hotspots & Heatmap

Aggregate workload requests and render a resource heatmap to expose CPU and memory pressure.

[Spot the nodes and VMs running hot at a glance.](#)

How · Web: Heatmap page. CLI: `veyron trends-analyze cpu_usage --window 24`, `veyron capacity --detailed`.



SLOs & Alerts

Track fleet availability SLO objectives and read namespace-aware warning-event narratives and alerts.

[Turn raw Kubernetes events into an availability story you can act on.](#)

How · CLI: `veyron slo`, `veyron alerts-list`. Web: SLO / Alerts pages.



Stack Discovery

Auto-detect the observability stack — Prometheus, Grafana, Loki, Jaeger — and deep-link into it.

[Jump from Veyron straight into the monitoring tools you already run.](#)

How · Web: Stack Health / Observability pages (deep-links). CLI: `veyron observability`.

GitOps & Operator

CR-native VMs, drift detection, and infrastructure-as-code delivery.



VeyronVM Operator

A Go controller-runtime operator reconciles VeyronVM and VeyronBlueprint custom resources into live KubeVirt VMs.

Manage VMs the Kubernetes-native way — declare them and let the operator converge.

How · CRD: `kubectl apply -f a VeyronVM / VeyronBlueprint ( veyron.io/v1alpha1 )`; install the operator from `charts/veyron-operator`.



GitOps Export

Export every VM in a namespace as VeyronVM CRD manifests with a single command.

Capture a running fleet as version-controlled YAML for Argo CD or Flux.

How · CLI: `veyron gitops-export --directory gitops/` (also `veyron export`). Commit the manifests to your GitOps repo.



Drift Detection

Compare live VMs against their desired CRD state and surface drift as badges, filters, and operator insights.

Know the moment a VM diverges from what Git says it should be.

How · CLI: `veyron gitops-diff --directory gitops/`, `veyron gitops-status`, `veyron drift`. Web: GitOps page (drift badges).



Helm & Terraform

Install via Helm charts and provision through a dedicated Terraform provider.

Fit Veyron into whatever IaC pipeline your platform team already uses.

How · Helm: `helm install` from `charts/veyron-operator`. Terraform: the provider in `terraform-provider-vmroque/`.



Catalog CRDs

Publish embedded templates and profiles to the cluster as VMTemplate/VMProfile CRDs and keep them in sync.

One authoritative, GitOps-managed catalog shared by CLI, operator, and dashboard.

How · CLI: `veyron catalog` to view; publish VMTemplate/VMProfile CRDs to the cluster (`kubectl apply`) so CLI, operator, and dashboard share names. Web: Catalog page.

Platform & Surfaces

One Rust core, four ways to drive it, across many clusters and tenants.

Surface	Best for
Web dashboard (CloudOS)	Visual operations, consoles, guided fixes
CLI (veyron)	Scripting, automation, CI/CD pipelines
TUI	Fast keyboard-driven cluster navigation
REST / WebSocket API	Custom tooling and integrations



CloudOS Dashboard

A macOS-style Mission Control web shell with 40+ pages, Spotlight (⌘K), a dock, Finder sidebar, and Fix-it flows.

A polished, discoverable home for operators who'd rather click than memorize flags.

How · Run `veyron serve`, then open `https://localhost:8080` (redirects to `/dashboard`); Spotlight with `⌘K`.



TUI & CLI

A k9s-style terminal UI with themes and dialogs, plus a full-featured headless CLI for scripting.

Fast keyboard-driven control at the terminal, and automatable commands for pipelines.

How · TUI: `veyron tui` (`--theme dark`, `--no-splash`, `--basic`). CLI: the `veyron` binary — `veyron commands` lists the full surface.



REST & WebSocket API

A Rust Axum API server (49 routes) with embedded OpenAPI and WebSocket console tunnels.

Build your own tooling on the same API the dashboard and CLI use.

How · Start with `veyron serve`; inspect with `veyron api-routes` and `veyron api-spec --format yaml`. Base path `/api/v1`; OpenAPI at `/api/openapi.json`.



Multi-Cluster & Tenancy

Switch between kubeconfig contexts from a cluster bar and carve the fleet into tenant workspaces with quotas and billing tags.

Operate many clusters and isolate teams from a single control plane.

How · CLI: `veyron clusters-list`, `veyron clusters-discover`, `veyron quotas`. Web: cluster bar + Quotas/Tenancy pages.



Automation & Integrations

Define automation rules (manual, schedule, event, metric triggers) and probe optional integrations from one page.

[Codify routine day-2 responses and see which backends are wired at a glance.](#)

How · CLI: `veyron webhook-list` / `veyron webhook-create` . Web: Automation / Integrations pages.

Support & next steps

Support & contact

Veyron is developed by **ZyvorAI Labs**. For onboarding, a proof-of-concept, or a guided demo, contact your account team at **info@zyvor.dev**.

Good to know: Veyron is a control plane over KubeVirt and Kubernetes and depends on those APIs being present. Several dashboard pages are honest about their limits: costs are estimates unless OpenCost is wired, security posture is configuration-derived rather than scanner-backed, and forecasting, observability counts, and scheduling metrics are heuristic. Optional integrations (OpenCost, Trivy, Prometheus, PacketWolf, Elastic/SIEM, an OpenAI-compatible LLM) unlock deeper data when configured.

Veyron

Declarative VM provisioning, day-2 operations, and a Mission
Control dashboard for KubeVirt — forged in Rust.

info@zyvor.dev · zyvor.dev